GNANAMANI COLLEGE OF TECHNOLOGY
(An Autonomous Institution)
Affiliated to Anna University - Chennai, Approved by AICTE - New Delhi.
(Accredited by NBA & NAAC with "A" Grade)
NH-7, A.K.SAMUTHIRAM, PACHAL (PO), NAMAKKAL - 637018.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

INSTITUTE

VISION

Emerging as a technical institution of high standard and excellence to produce quality Engineers, Researchers, Administrators and Entrepreneurs with ethical and moral values to contribute the sustainable development of the society.

MISSION

We facilitate our students

MI: To have in-depth domain knowledge with analytical and practical skills in cutting edge technologies by imparting quality technical education.

MII: To be industry ready and multi-skilled personalities to transfer technology to industries and rural areas by creating interests among students in Research and Development and Entrepreneurship.

DEPARTMENT

VISION

To be a Centre of Artificial Intelligence and Data Science by imparting quality education, promoting research and innovation with global relevance.

MISSION

- To impart holistic education with niche technologies for the enrichment of knowledge and skills through updated curriculum and inspired learning.
- To empower valued based AI education to the students for developing intelligent systems and innovative products to address the societal problems with ethical value.
- To work in close liaison with industry to achieve socio-economic development.

GNANAMANI COLLEGE OF TECHNOLOGY
(An Autonomous Institution)
Affiliated to Anna University - Chennai, Approved by AICTE - New Delhi.
(Accredited by NBA & NAAC with "A" Grade)
NH-7, A.K.SAMUTHIRAM, PACHAL (PO), NAMAKKAL - 637018.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

PROGRAM OUTCOMES (POs)

PO-1 **Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.

PO-2 **Problem Analysis:** Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO-3 **Design/Development of Solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO-4 **Conduct Investigations of Complex Problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO-5 **Modern Tool Usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO-6 **The Engineer and Society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO-7 **Environment and Sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO-8 **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO-9 **Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO-10 **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO-11 **Project Management and Finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO-12 **Life-Long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

GNANAMANI COLLEGE OF TECHNOLOGY
(An Autonomous Institution)
Affiliated to Anna University - Chennai, Approved by AICTE - New Delhi.
(Accredited by NBA & NAAC with "A" Grade)
NH-7, A.K.SAMUTHIRAM, PACHAL (PO), NAMAKKAL - 637018.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

The Programme Educational Objectives of B.Tech (Artificial Intelligence and Data Science) are listed below:

PEO-1: Our Graduates are competent in building intelligent machines, software, or applications with a cutting-edge combination of machine learning, analytics and visualization technologies to identify new opportunities.

PEO-2: Our Graduates adapt the new technologies and develop the solutions to real world problems with ethical practices to enhance their own stature to contribute society.

PEO-3: Our Graduates thrive to continuing education for fulfilling their lifelong goals and satisfaction and successful professionals in industry, government, academia, research and consultancy.

PROGRAM SPECIFIC OUTCOMES (PSOs)

Engineering Graduates will be able to:

PSO-1: Apply the fundamental knowledge to develop intelligent systems using computational principles, methods and systems for extracting knowledge from data to identify, formulate and solve real time problems and societal issues for the sustainable development.

PSO-2: Enrich their abilities to qualify for Employment, Higher studies and Research in Artificial Intelligence and Data science with ethical values.

Experiment 1 **Solution to XOR problem using DNN**

Date

AIM:

To solve XOR problem using DNN

CONCEPT:

The XOR problem is a classic problem in the field of machine learning and artificial intelligence. XOR stands for "exclusive OR," which is a logical operation that takes two binary inputs (0 or 1)and returns 1 only when exactly one of the inputs is 1. Otherwise, it returns 0. Here's the truthtable for the XOR operation:

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

For solving the XOR problem, use a Deep Neural Network (DNN) implemented with TensorFlow and Keras. The XOR problem is a classic binary classification problem thatcannot be linearly separated. A DNN with hidden layers can learn the non-linear patternsrequired to solve this problem.

1. **Training:**
   - Provide the input data (XOR inputs) and the corresponding labels (XOR outputs) tothe DNN.
   - The DNN adjusts its internal weights through backpropagation to minimize the lossfunction.
   - The model learns to approximate the XOR function based on the providedexamples.
2. **Epochs and Batch Size:**
   - Train the model for multiple epochs (iterations over the entire dataset) to allow theDNN to adjust its weights and learn the XOR function.
   - You can choose a suitable batch size for updating weights in each iteration.
3. **Evaluation:**
   - After training, evaluate the model's performance using the same XOR data.
   - Calculate metrics like accuracy to assess how well the model has learned the XORfunction.
4. **Inference:**
   - Once trained, the DNN can be used to predict the XOR output for new inputcombinations.

STEPS

1	Define the XOR input and outputdata	X: Input data for the XOR problem, where each row represents aninput sample. y: Corresponding target outputs for the XOR problem.
2	Define the DNN architecture	Input Layer: The input layer specifies an input shape of (2,), indicating thatthe model expects input data with two features. Hidden Layer 1 (Dense): This hidden layer has 8 units/neurons. It uses the ReLU activation function ('relu'), which introduces non-linearity to the network. Hidden Layer 2 (Dense): This hidden layer also has 8 units/neurons.It also uses the ReLU activation function. Output Layer (Dense): The output layer has 1 unit/neuron, which corresponds to thebinary classification task. It uses the sigmoid activation function ('sigmoid') to produce aprobability score between 0 and 1- indicating likelihood of positive class
3	Compile the model	model.compile(loss='mean_squared_error', optimizer='adam'): Configures the model for training. It specifies the loss function ('mean_squared_error') and the optimization algorithm ('adam').
4	Train the DNN	**model.fit(X, y, epochs=10000, verbose=0)**: Trains the model on the XOR dataset. The **epochs** parameter specifies the number of timesthe dataset is iterated during training. **verbose=0** means that no progress updates will be printed during training.
5	Test the trained DNN	• **predictions = model.predict(X)**: Uses the trained model tomake predictions on the same XOR dataset. • **print(predictions)**: Prints the predicted outputs for the XORinputs.

PROGRAM

```
import tensorflow as tfimport
numpy as np

# Define the XOR input and output data
x_data = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=np.float32)
y_data = np.array([[0], [1], [1], [0]], dtype=np.float32)

# Define the DNN architecturemodel =
tf.keras.Sequential([
        tf.keras.layers.Dense(8, input_dim=2, activation='relu'),
        tf.keras.layers.Dense(8, activation='relu'), tf.keras.layers.Dense(1,
        activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy',metrics=['accuracy'])

# Train the DNN
model.fit(x_data, y_data, epochs=1000, verbose=0)

# Test the trained DNN
predictions = model.predict(x_data) rounded_predictions =
np.round(predictions)print("Predictions:", rounded_predictions)
```

OUTPUT

```
Predictions: [[0.]
 [1.]
 [1.]
 [0.]]
```

RESULT

Thus XOR problem using DNN is solved.

Experiment 2 **Character recognition using CNN**

Date

AIM:

To implement character Recognition using CNN

CONCEPT:

Character recognition using Convolutional Neural Networks (CNNs) is a widely used technique in the field of computer vision and machine learning. CNNs are particularly well- suited for tasks like character recognition because they can automatically learn hierarchical features from the input data, capturing both local and global patterns.

1. **Data Preparation**: Collect and preprocess a dataset of labeled character images. Each image should be associated with a label indicating the corresponding character. You may need to resize, normalize, and augment the images to improve the model's performance and generalization.
2. **Model Architecture**: Design a CNN architecture for character recognition. A typical CNN architecture consists of alternating convolutional and pooling layers, followed by one or more fully connected layers. The convolutional layers extract features from the input images, while the pooling layers downsample the feature maps to reduce dimensionality.
3. **Convolutional and Pooling Layers**: Convolutional layers consist of multiple filters that slide over the input image, computing dot products between the filter weights and local image patches. This process captures local patterns and features. Pooling layers (e.g., max pooling) reduce the spatial dimensions of the feature maps, retaining the most salient information.
4. **Fully Connected Layers**: After the convolutional and pooling layers, flatten the feature maps and feed them into one or more fully connected layers. These layers perform classification based on the learned features. The output layer should have as many neurons as there are classes (characters), and a suitable activation function (e.g., softmax) is applied to produce class probabilities.
5. **Training**: Initialize the CNN's weights randomly and use a labeled training dataset to optimize these weights. Employ a loss function (e.g., categorical cross-entropy) to measure the difference between predicted and actual labels. Backpropagate the error and update the weights using an optimization algorithm (e.g., stochastic gradient descent or Adam) to minimize the loss.
6. **Validation and Testing**: Split your dataset into training, validation, and testing sets. Monitor the model's performance on the validation set during training to prevent overfitting. Once training is complete, evaluate the model's accuracy on the testing set to assess its real-world performance.
7. **Hyperparameter Tuning**: Experiment with various hyperparameters such as learning rate, batch size, number of layers, filter sizes, and pooling strategies to optimize the model's performance.
8. **Deployment**: Once satisfied with the model's performance, deploy it to recognize characters in new, unseen images. You can integrate the trained CNN into applications

such as optical character recognition (OCR) systems, document processing pipelines, or other relevant projects.

Keep in mind that the success of your character recognition system will depend onfactors like the quality and quantity of your dataset, the architecture of your CNN, andthe effectiveness of your training process. Additionally, there are pre-trained CNN architectures, such as VGG, ResNet, and Inception, which you can fine-tune for character recognition tasks to potentially achieve better performance.

STEPS

1. Load the MNIST dataset
2. Preprocess the data
3. Define the CNN architecture
4. Compile, train and evaluate the model

PROGRAM

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist from
tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Preprocess the data
x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0
x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0

# Define the CNN architecturemodel =
Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Flatten()) model.add(Dense(64,
activation='relu'))
model.add(Dense(10, activation='softmax'))

# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',metrics=['accuracy'])

# Train the CNN
model.fit(x_train, y_train, epochs=5, batch_size=64, validation_data=(x_test,y_test))
```

```
# Evaluate the model
loss, accuracy = model.evaluate(x_test, y_test)print('Test Loss:',
loss)
print('Test Accuracy:', accuracy)
```

OUTPUT

```
Epoch 1/5
938/938 [==============================] - 21s 21ms/step - loss: 0.1687 - accuracy: 0.
9493 - val_loss: 0.0792 - val_accuracy: 0.9755
Epoch 2/5
938/938 [==============================] - 19s 21ms/step - loss: 0.0511 - accuracy: 0.
9847 - val_loss: 0.0426 - val_accuracy: 0.9855
Epoch 3/5
938/938 [==============================] - 20s 21ms/step - loss: 0.0365 - accuracy: 0.
9884 - val_loss: 0.0308 - val_accuracy: 0.9900
Epoch 4/5
938/938 [==============================] - 20s 21ms/step - loss: 0.0274 - accuracy: 0.
9915 - val_loss: 0.0319 - val_accuracy: 0.9889
Epoch 5/5
938/938 [==============================] - 20s 21ms/step - loss: 0.0230 - accuracy: 0.
9927 - val_loss: 0.0353 - val_accuracy: 0.9901
313/313 [==============================] - 1s 4ms/step - loss: 0.0353 - accuracy: 0.99
01
Test Loss: 0.03527578338980675
Test Accuracy: 0.9901000261306763
```

RESULT

Thus character Recognition using CNN is implemented.

Experiment 3 **Face recognition using CNN**

Date

AIM:

To Implement Face recognition using CNN

CONCEPT:

Face recognition using Convolutional Neural Networks (CNNs) is a popular application of deep learning in computer vision. CNNs are well-suited for image recognition tasks like face recognition because they can automatically learn hierarchical features from images, whichare essential for discriminating between different faces.

Here's an overview of how to approach face recognition using CNNs:

1. **Data Collection and Preprocessing:**
   - Collect a dataset of labeled face images. You need images of individuals you want the model to recognize.
   - Preprocess the images: Resize them to a common size (e.g., 224x224 pixels),normalize pixel values to a certain range (usually [0, 1] or [-1, 1]), and perform data augmentation (e.g., random cropping, flipping) to increase model robustness.
2. **CNN Architecture:**
   - Choose a CNN architecture: Common choices include variants of VGG, ResNet, Inception, or MobileNet. These architectures are available in libraries like TensorFlow and PyTorch.
   - Pretrained Models: Consider using a pretrained CNN model on a large dataset like ImageNet. Transfer learning allows you to leverage learned features and fine-tune the model for face recognition.
3. **Model Training:**
   - Modify the architecture: Replace the classification head of the pretrained model with a new one suitable for face recognition. Typically, the new head consists of afew fully connected layers.
   - Data Input: Feed the preprocessed face images into the model and train it using a suitable loss function. Common choices include triplet loss or contrastive loss, which encourage the network to learn embeddings that make similar faces close and dissimilar faces far apart in the embedding space.
   - Optimization: Use an optimizer like Adam or SGD to update the model's parameters during training.
4. **Testing and Verification:**
   - Embedding Extraction: After training, remove the classification head and use the model to extract embeddings (feature vectors) from face images.
   - Face Verification: To verify whether two face images belong to the same person, calculate the similarity (e.g., cosine similarity) between their embeddings. Define athreshold to decide whether the faces are a match or not.
   - Face Identification: For identification, compare the embeddings of a query face against embeddings of all known faces in the database and find the closest

match.

5. **Deployment and Testing:**
   - Deploy the trained model to your desired platform (web application, mobile app,etc.).
   - Test the model's accuracy and performance on real-world data. Tweakhyperparameters or augment the dataset as needed to improve accuracy.

Remember that face recognition involves privacy concerns, and ethical considerations shouldbe taken into account, especially when working with sensitive data.

STEPS

1 Set the path to the directory containing the face images

2 Load the face images and labels & Iterate over the face image directory and load the images

3 Convert the data to numpy arrays, Preprocess labels to extract numeric partAnd Convert labels to one-hot encoded vectors

4 Split the data into training and validation sets

5 Compile, Train the CNN model and Save the trained model

PROGRAM

```
import os
import numpy as np import
tensorflow as tf
from tensorflow.keras.preprocessing.image import load_img, img_to_arrayfrom
sklearn.model_selection import train_test_split

# Set the path to the directory containing the face imagesfaces_dir = "D:/R2021 DL
LAB/Faces/Faces"

# Load the face images and labelsx_data = []
y_data = []

# Iterate over the face image directory and load the imagesfor filename in
os.listdir(faces_dir):
    if filename.endswith(".jpg"):
        img_path = os.path.join(faces_dir, filename)
        img = load_img(img_path, target_size=(64, 64)) # Resize images to64x64 pixels
        img_array = img_to_array(img)x_data.append(img_array)
        label = filename.split(".")[0]
# Assuming the filename format is label.jpgy_data.append(label)

# Convert the data to numpy arrays
```

```
x_data = np.array(x_data)y_data
= np.array(y_data)

# Preprocess labels to extract numeric part
y_data_numeric = np.array([int(label.split("_")[1]) for label in y_data])

# Convert labels to one-hot encoded vectors num_classes =
len(np.unique(y_data_numeric))
y_data_encoded = tf.keras.utils.to_categorical(y_data_numeric, num_classes)

# Split the data into training and validation sets
x_train, x_val, y_train, y_val = train_test_split(x_data, y_data_encoded,test_size=0.2, random_state=42)

# Define the CNN architecture for face recognitionmodel =
tf.keras.models.Sequential([
        tf.keras.layers.Conv2D(32, (3, 3), activation='relu', input_shape=(64, 64,
3)),
        tf.keras.layers.MaxPooling2D((2, 2)), tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128,  activation='relu'),
        tf.keras.layers.Dense(num_classes,  activation='softmax')

])

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy',metrics=['accuracy'])

# Train the CNN model
model.fit(x_train, y_train, epochs=10, batch_size=32, validation_data=(x_val,y_val))

# Save the trained model model.save("face_recognition_model.keras")
```

OUTPUT

```
Epoch 1/10
65/65 [==============================] - 5s 68ms/step - loss: 215.6209 - accuracy: 0.0
098 - val_loss: 4.7830 - val_accuracy: 0.0039
Epoch 2/10
65/65 [==============================] - 4s 66ms/step - loss: 4.7793 - accuracy: 0.011
2 - val_loss: 4.7757 - val_accuracy: 0.0039Epoch 3/10
65/65 [==============================] - 4s
                                    66ms/step - loss: 4.7717 - accuracy: 0.012
2 - val_loss: 4.7694 - val_accuracy: 0.0039Epoch 4/10
65/65 [==============================] - 4s
                                    66ms/step - loss: 4.7646 - accuracy: 0.010
7 - val_loss: 4.7634 - val_accuracy: 0.0039
Epoch 5/10
```

65/65 [==============================] - 4s 68ms/step - loss: 4.7579 - accuracy: 0.010
2 - val_loss: 4.7577 - val_accuracy: 0.0039
Epoch 6/10
65/65 [==============================] - 5s 70ms/step - loss: 4.7516 - accuracy: 0.010
7 - val_loss: 4.7525 - val_accuracy: 0.0039
Epoch 7/10
65/65 [==============================] - 4s 66ms/step - loss: 4.7457 - accuracy: 0.009
8 - val_loss: 4.7476 - val_accuracy: 0.0039
Epoch 8/10
65/65 [==============================] - 4s 67ms/step - loss: 4.7402 - accuracy: 0.010
7 - val_loss: 4.7432 - val_accuracy: 0.0039
Epoch 9/10
65/65 [==============================] - 4s 66ms/step - loss: 4.7349 - accuracy: 0.013
2 - val_loss: 4.7392 - val_accuracy: 0.0078
Epoch 10/10
65/65 [==============================] - 4s 65ms/step - loss: 4.7300 - accuracy: 0.010
2 - val_loss: 4.7354 - val_accuracy: 0.0078

CHECK OUTPUT AS IMAGES IN MENTIONED FOLDER

RESULT

Face recognition using CNN is analysed and implemented.

Experiment 4 **Language modeling using RNN**

Date

AIM:

To implement Language modeling using RNN

CONCEPT:

Language modeling is a fundamental task in natural language processing (NLP) that involves predicting the next word or character in a sequence of text. Recurrent Neural Networks (RNNs) are a class of neural networks commonly used for language modeling dueto their ability to capture sequential dependencies in data.

1. **Data Preparation:** Language modeling requires a large dataset of text. This could be a collection of sentences, paragraphs, or entire documents. The text is usually tokenized into words or characters, and each token is assigned a unique integer ID.
2. **Creating Input-Output Pairs:** The next step is to create input-output pairs for training. Foreach token in the dataset, the input will be the previous sequence of tokens, and theoutput will be the token that follows. For example, given the sentence "I love to eat ice," the pairs would be: ("I love to eat", "ice"), ("love to eat ice", "cream"), and so on.
3. Embedding Layer: Each token ID is typically transformed into a dense vector representation using an embedding layer. This helps the network learn meaningful representations of words or characters.
4. **RNN Architecture:** The core of the language model is the RNN layer. At each time step, the RNN takes in the current token's embedding and the previous hidden state, andproduces a new hidden state. This hidden state captures the context of the previoustokens in the sequence.
5. **Output Layer:** The output of the RNN at each time step is used to predict the probability distribution over the vocabulary (i.e., all possible tokens). This is typically done using a softmax activation function. The token with the highest predicted probability is the predicted next token.
6. **Training:** The model is trained to minimize the cross-entropy loss between the predicted probability distribution and the actual next token. This involves adjusting the model's weights using backpropagation through time (BPTT).
7. **Sampling:** Once the model is trained, you can use it to generate new text. To do this, youprovide an initial seed sequence, and then repeatedly sample from the predicted probability distribution to generate the next token. This token is then fed back into the model as input for the next time step, and the process continues to generate a sequenceof desired length.
8. **Hyperparameter Tuning**: There are several hyperparameters to tune, such as the numberof hidden units in the RNN, the type of RNN cell (e.g., LSTM, GRU), the sequence length, learning rate, and more. Tuning these hyperparameters can significantly impact theperformance of the language model.

It's worth noting that while RNNs were popular for language modeling, more advanced architectures like LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) have been

developed to address some of the limitations of traditional RNNs, such as the vanishing gradientproblem.
Additionally, modern language models like GPT (Generative Pre-trained Transformer) havelargely surpassed the performance of traditional RNN-based models by utilizing attention mechanisms and larger-scale architectures.

STEPS

1. Create a set of unique characters in the text
2. Convert text to a sequence of character indices
3. Create input-output pairs for training
4. Convert sequences and next_char to numpy arrays
5. Train the model and Generate text using the trained model

PROGRAM

```
import numpy as np import
tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

# Sample text data
text = "This is a sample text for language modeling using RNN."

# Create a set of unique characters in the textchars =
sorted(set(text))
char_to_index = {char: index for index, char in enumerate(chars)}index_to_char = {index: char for
index, char in enumerate(chars)}

# Convert text to a sequence of character indices text_indices =
[char_to_index[char] for char in text]

# Create input-output pairs for trainingseq_length = 20
sequences = []next_char = []
for i in range(0, len(text_indices) - seq_length):
        sequences.append(text_indices[i : i + seq_length])
        next_char.append(text_indices[i + seq_length])

# Convert sequences and next_char to numpy arraysX =
np.array(sequences)
y = np.array(next_char)

# Build the RNN model
```

```
model = Sequential([
    Embedding(input_dim=len(chars), output_dim=50, input_length=seq_length),SimpleRNN(100,
    return_sequences=False),
    Dense(len(chars), activation="softmax")
])

model.compile(loss="sparse_categorical_crossentropy", optimizer="adam")# Train the model
model.fit(X, y, batch_size=64, epochs=50)

# Generate text using the trained modelseed_text =
"This is a sample te" generated_text = seed_text
num_chars_to_generate = 100

for _ in range(num_chars_to_generate):
    seed_indices = [char_to_index[char] for char in seed_text]

    # Check if the seed sequence length matches the model's input lengthif len(seed_indices) <
    seq_length:
        diff = seq_length - len(seed_indices) seed_indices = [0]
        * diff + seed_indices

    seed_indices = np.array(seed_indices).reshape(1, -1)next_index =
    model.predict(seed_indices).argmax() next_char =
    index_to_char[next_index]
    generated_text += next_char
    seed_text = seed_text[1:] + next_charprint(generated_text)
```

OUTPUT

```
Epoch 1/50
1/1 [==============================] - 1s 1s/step - loss: 3.0885
Epoch 2/50
1/1 [==============================] - 0s 8ms/step - loss: 3.0053
Epoch 3/50
1/1 [==============================] - 0s 14ms/step - loss: 2.9234
Epoch 4/50
1/1 [==============================] - 0s 0s/step - loss: 2.8392
Epoch 5/50
1/1 [==============================] - 0s 17ms/step - loss: 2.7501
Epoch 6/50
1/1 [==============================] - 0s 0s/step - loss: 2.6545
Epoch 7/50
1/1 [==============================] - 0s 4ms/step - loss: 2.5519
Epoch 8/50
```

1/1 [==============================] - 0s 14ms/step - loss: 2.4425
Epoch 9/50
1/1 [==============================] - 0s 0s/step - loss: 2.3266
Epoch 10/50
1/1 [==============================] - 0s 18ms/step - loss: 2.2063
Epoch 11/50
1/1 [==============================] - 0s 8ms/step - loss: 2.0865
Epoch 12/50
1/1 [==============================] - 0s 5ms/step - loss: 1.9717
Epoch 13/50
1/1 [==============================] - 0s 0s/step - loss: 1.8622
Epoch 14/50
1/1 [==============================] - 0s 4ms/step - loss: 1.7552
Epoch 15/50
1/1 [==============================] - 0s 13ms/step - loss: 1.6493
Epoch 16/50
1/1 [==============================] - 0s 0s/step - loss: 1.5457
Epoch 17/50
1/1 [==============================] - 0s 17ms/step - loss: 1.4472
Epoch 18/50
1/1 [==============================] - 0s 0s/step - loss: 1.3554
Epoch 19/50
1/1 [==============================] - 0s 17ms/step - loss: 1.2678
Epoch 20/50
1/1 [==============================] - 0s 0s/step - loss: 1.1810
Epoch 21/50
1/1 [==============================] - 0s 17ms/step - loss: 1.0964
Epoch 22/50
1/1 [==============================] - 0s 14ms/step - loss: 1.0179
Epoch 23/50
1/1 [==============================] - 0s 1ms/step - loss: 0.9459
Epoch 24/50
1/1 [==============================] - 0s 16ms/step - loss: 0.8773
Epoch 25/50
1/1 [==============================] - 0s 0s/step - loss: 0.8107
Epoch 26/50
1/1 [==============================] - 0s 17ms/step - loss: 0.7473
Epoch 27/50
1/1 [==============================] - 0s 0s/step - loss: 0.6884

```
Epoch 28/50
1/1 [==============================] - 0s 17ms/step - loss: 0.6333
Epoch 29/50
1/1 [==============================] - 0s 0s/step - loss: 0.5809
Epoch 30/50
1/1 [==============================] - 0s 2ms/step - loss: 0.5318
```

```
Epoch 31/50
1/1 [==============================] - 0s 17ms/step - loss: 0.4871
Epoch 32/50
1/1 [==============================] - 0s 0s/step - loss: 0.4469
Epoch 33/50
1/1 [==============================] - 0s 18ms/step - loss: 0.4099
Epoch 34/50
1/1 [==============================] - 0s 0s/step - loss: 0.3753
Epoch 35/50
1/1 [==============================] - 0s 18ms/step - loss: 0.3430
Epoch 36/50
1/1 [==============================] - 0s 0s/step - loss: 0.3134
Epoch 37/50
1/1 [==============================] - 0s 15ms/step - loss: 0.2865
Epoch 38/50
1/1 [==============================] - 0s 0s/step - loss: 0.2621
Epoch 39/50
1/1 [==============================] - 0s 2ms/step - loss: 0.2399
Epoch 40/50
1/1 [==============================] - 0s 15ms/step - loss: 0.2200
Epoch 41/50
1/1 [==============================] - 0s 1ms/step - loss: 0.2021
Epoch 42/50
1/1 [==============================] - 0s 18ms/step - loss: 0.1860
Epoch 43/50
1/1 [==============================] - 0s 0s/step - loss: 0.1714
Epoch 44/50
1/1 [==============================] - 0s 16ms/step - loss: 0.1580
Epoch 45/50
1/1 [==============================] - 0s 0s/step - loss: 0.1460
Epoch 46/50
1/1 [==============================] - 0s 4ms/step - loss: 0.1353
Epoch 47/50
1/1 [==============================] - 0s 12ms/step - loss: 0.1257
Epoch 48/50
1/1 [==============================] - 0s 933us/step - loss: 0.1170
Epoch 49/50
1/1 [==============================] - 0s 17ms/step - loss: 0.1090
Epoch 50/50
```

1/1 [==============================] - 0s 0s/step - loss: 0.1017

This is a sample tentrfornlanguags modnging nsing Rgn.rginsrngangrngangnoggrng nsingrngingndgg nsinorng ngrngadgsinorng

RESULT

Thus Language modeling using RNN is implemented.

Experiment 5 **Sentiment analysis using LSTM**

Date

AIM:

To implement Sentiment analysis using LSTM

CONCEPT:

Sentiment analysis using LSTM (Long Short-Term Memory) is a common task in natural language processing where you aim to determine the sentiment or emotional tone expressed in a given text. LSTM is a type of **recurrent neural network** that can capture long-range dependencies in sequences, making it well-suited for sequence-based tasks like sentiment analysis.

1. **Text Representation**: In sentiment analysis, text data needs to be converted into a numerical format that can be processed by neural networks. This is typically done using techniques like tokenization and word embedding. Tokenization splits the text into individual words or subwords, while word embedding maps each token to a dense vector representation in a continuous vector space.
2. **Sequence Padding**: In order to train LSTM networks efficiently, sequences (sentences or documents) need to have a consistent length. Since text data can have varying lengths,padding is applied to make all sequences of the same length. Shorter sequences arepadded with zeros at the beginning or end.
3. **LSTM Architecture**: An LSTM is a type of recurrent neural network designed to handlesequential data. It has memory cells and gates that allow it to capture long-term dependencies in sequences. LSTMs can remember important information over extended periods, which is crucial for sentiment analysis since sentiments in text can span across multiple words.
4. **Embedding Layer**: The input text tokens are passed through an embedding layer, which converts the discrete token indices into dense vector representations. This layer is responsible for capturing semantic relationships between words.
5. **LSTM Layer**: The LSTM layer processes the embedded sequences, updating its internal state based on the input tokens and previous state. The LSTM's ability to maintain and update context over time enables it to capture sequential patterns and dependencieswithin the text.
6. **Classification Layer**: After processing the sequence through the LSTM layer, the final hidden state is passed through a fully connected (dense) layer. This layer performs the sentiment classification by producing a probability score indicating the likelihood of aparticular sentiment class.
7. **Training and Backpropagation**: During training, the model's predictions are compared tothe actual sentiment labels using a loss function (such as binary cross-entropy for binarysentiment classification). The gradients of the loss are propagated back through the network using backpropagation through time (BPTT), and the model's parameters areupdated using an optimization algorithm (e.g., Adam, SGD).
8. **Inference and Prediction**: Once the LSTM model is trained, it can be used to predict sentiment labels for new, unseen text data. The input text is processed through the trained

model, and the final classification layer's output provides the predicted sentiment.

Sentiment analysis using LSTM is a powerful application of deep learning in natural language processing. It allows the model to learn and capture complex patterns in textual data, making it capable of understanding and classifying sentiments expressed in various contexts.

STEPS

1. Load the IMDB dataset, which consists of movie reviews labeled with positive or negative sentiment.
2. Preprocess the data by padding sequences to a fixed length (**max_review_length**)and limiting the vocabulary size to the most frequent words (**num_words**).
3. Build an LSTM-based model. The Embedding layer is used to map word indices to dense vectors, the LSTM layer captures sequence dependencies, and the Dense layerproduces a binary sentiment prediction.
4. The model is compiled with binary cross-entropy loss and the Adam optimizer.
5. Train the model using the training data. Finally, we evaluate the model on the testdata and print the test accuracy.

PROGRAM

```
import numpy as np import
tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing import sequencefrom
tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# Load the IMDb movie review dataset
max_features = 5000  # Number of words to consider as features
max_len = 500 # Maximum length of each review (pad shorter reviews, truncatelonger reviews)
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)x_train =
sequence.pad_sequences(x_train, maxlen=max_len)
x_test = sequence.pad_sequences(x_test, maxlen=max_len)

# Define the LSTM model
embedding_size = 32  # Dimensionality of the word embeddings

model = Sequential()
model.add(Embedding(max_features, embedding_size, input_length=max_len))
model.add(LSTM(100)) # LSTM layer with 100 units
model.add(Dense(1, activation='sigmoid'))
```

```
# Compile the model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

# Train the modelbatch_size = 64
epochs = 5

model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs,validation_data=(x_test, y_test))

# Evaluate the model
loss, accuracy = model.evaluate(x_test, y_test)print("Loss:",
loss)
print("Accuracy:", accuracy)
```

OUTPUT

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb
.npz
17464789/17464789 [==============================] - 7s 0us/step
Epoch 1/5
391/391 [==============================] - 286s 727ms/step - loss: 0.4991 - accuracy:
0.7626 - val_loss: 0.3712 - val_accuracy: 0.8412Epoch 2/5
391/391 [==============================] - 296s 757ms/step - loss: 0.3381 - accuracy:
0.8587 - val_loss: 0.3609 - val_accuracy: 0.8532Epoch 3/5
391/391 [==============================] - 313s 801ms/step - loss: 0.2642 - accuracy:
0.8945 - val_loss: 0.3168 - val_accuracy: 0.8678Epoch 4/5
391/391 [==============================] - 433s 1s/step - loss: 0.2263 - accuracy: 0.9
142 - val_loss: 0.3119 - val_accuracy: 0.8738Epoch 5/5
391/391 [==============================] - 302s 774ms/step - loss: 0.1982 - accuracy:
0.9247 - val_loss: 0.3114 - val_accuracy: 0.8745
782/782 [==============================] - 74s 95ms/step - loss: 0.3114 - accuracy: 0.
8745
Loss: 0.3113741874694824
Accuracy: 0.8745200037956238
```

RESULT

Thus Sentiment analysis using LSTM is implemented.

Experiment 6 **Parts of speech tagging using Sequence to Sequence architecture** **Date**

AIM:

To implement Parts of speech tagging using Sequence to Sequence architecture

CONCEPT:

Parts of speech (POS) tagging is a natural language processing task where each word in a sentence is assigned a specific grammatical category, such as noun, verb, adjective, etc. Sequence-to-Sequence (Seq2Seq) architecture, which was originally designed for machine translation tasks, can also be adapted for POS tagging. The Seq2Seq architecture consists of two main components: an encoder and a decoder. Here's how you can use Seq2Seq for POS tagging:

1. **Encoder-Decoder Setup**: In the context of POS tagging, the encoder takes in the input sentence (sequence of words) and encodes it into a fixed-size context vector. The decoder then generates the POS tags based on this context vector.
2. **Encoder Component**: The encoder can be implemented using a recurrent neural network (RNN) such as LSTM or GRU. The input sequence of words (tokens) is passed through the encoder RNN, and the final hidden state of the encoder captures the contextual information of the entire sentence.
3. **Decoder Component**: The decoder is another RNN that takes the encoder's final hidden state as an initial hidden state and generates POS tags one at a time. At each step, the decoder produces a probability distribution over possible POS tags for the current word.
4. **Training**: During training, the model is given pairs of input sentences and corresponding POS tag sequences. The encoder generates the context vector, which is then used as the initial state of the decoder. The decoder generates the predicted POS tags. The model is trained to minimize the cross-entropy loss between the predicted and actual POS tags.
5. **Inference**: During inference (testing or prediction), the model is given an input sentence, and the encoder generates the context vector. The decoder then generates POS tags one by one, using the context vector and the previously generated tag. This process continues until an end-of-sentence token is generated or a maximum sequence length is reached.

STEPS

1 Define the input and output sequences.

2 Create a set of all unique words and POS tags in the dataset

3 Add <sos> and <eos> tokens to target_words.

4 Create dictionaries to map words and POS tags to integers.

5 Define the maximum sequence lengths, Prepare the encoder input data And Prepare the decoder input and target data

6 Define the encoder input and LSTM layersDefine the decoder input and LSTM layers

7 Define, Compile and train the model

8 Define the encoder model to get the encoder states and Define the decoder model with encoder states as initial state

9 Define a function to perform inference and generate POS tags,Test the model.

PROGRAM

```
import numpy as np import
tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, LSTM, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Define the input and output sequences
input_texts = ['I love coding', 'This is a pen', 'She sings well']target_texts = ['PRP VB NNP', 'DT
VBZ DT NN', 'PRP VBZ RB']

# Create a set of all unique words and POS tags in the datasetinput_words = set()
target_words = set()
for input_text, target_text in zip(input_texts, target_texts):
    input_words.update(input_text.split()) target_words.update(target_text.split())

# Add <sos> and <eos> tokens to target_words
target_words.add('<sos>') target_words.add('<eos>')

# Create dictionaries to map words and POS tags to integers input_word2idx = {word: idx for
idx, word in enumerate(input_words)} input_idx2word = {idx: word for idx, word in
enumerate(input_words)} target_word2idx = {word: idx for idx, word in
enumerate(target_words)}target_idx2word = {idx: word for idx, word in
enumerate(target_words)}

# Define the maximum sequence lengths
max_encoder_seq_length = max([len(text.split()) for text in input_texts]) max_decoder_seq_length =
max([len(text.split()) for text in target_texts])

# Prepare the encoder input data
encoder_input_data = np.zeros((len(input_texts), max_encoder_seq_length),dtype='float32')
for i, input_text in enumerate(input_texts):
    for t, word in enumerate(input_text.split()): encoder_input_data[i, t] =
        input_word2idx[word]
```

```
# Prepare the decoder input and target data
decoder_input_data = np.zeros((len(input_texts), max_decoder_seq_length),dtype='float32')
decoder_target_data = np.zeros((len(input_texts), max_decoder_seq_length,len(target_words)),
dtype='float32')
for i, target_text   in   enumerate(target_texts):for t, word in
       enumerate(target_text.split()):
              decoder_input_data[i, t] = target_word2idx[word]if t > 0:
                     decoder_target_data[i, t - 1, target_word2idx[word]] = 1.0

# Define the encoder input and LSTM layersencoder_inputs =
Input(shape=(None,))
encoder_embedding = tf.keras.layers.Embedding(len(input_words),256)(encoder_inputs)
encoder_lstm = LSTM(256, return_state=True)
encoder_outputs, state_h, state_c = encoder_lstm(encoder_embedding)encoder_states = [state_h,
state_c]

# Define the decoder input and LSTM layersdecoder_inputs =
Input(shape=(None,))
decoder_embedding = tf.keras.layers.Embedding(len(target_words),256)(decoder_inputs)
decoder_lstm = LSTM(256, return_sequences=True, return_state=True)decoder_outputs, _, _ =
decoder_lstm(decoder_embedding, initial_state=encoder_states)
decoder_dense = Dense(len(target_words), activation='softmax')decoder_outputs =
decoder_dense(decoder_outputs)

# Define the model
model = Model([encoder_inputs, decoder_inputs], decoder_outputs)

# Compile and train the model
model.compile(optimizer='adam', loss='categorical_crossentropy',metrics=['accuracy'])
model.fit([encoder_input_data, decoder_input_data], decoder_target_data,batch_size=64,
epochs=50, validation_split=0.2)

# Define the encoder model to get the encoder states encoder_model =
Model(encoder_inputs, encoder_states)

# Define the decoder model with encoder states as initial statedecoder_state_input_h =
Input(shape=(256,)) decoder_state_input_c = Input(shape=(256,))
decoder_states_inputs = [decoder_state_input_h, decoder_state_input_c]decoder_outputs, state_h,
state_c = decoder_lstm(decoder_embedding,
```

```
initial_state=decoder_states_inputs) decoder_states = [state_h,
state_c] decoder_outputs = decoder_dense(decoder_outputs)
decoder_model = Model([decoder_inputs] + decoder_states_inputs, [decoder_outputs]
+ decoder_states)

# Define a function to perform inference and generate POS tagsdef
generate_pos_tags(input_sequence):
    states_value = encoder_model.predict(input_sequence)target_sequence
    = np.zeros((1, 1)) target_sequence[0, 0] = target_word2idx['<sos>']
    stop_condition = False
    pos_tags = []
    while not stop_condition:
        output_tokens, h, c = decoder_model.predict([target_sequence] +states_value)
        sampled_token_index = np.argmax(output_tokens[0, -1, :])sampled_word =
        target_idx2word[sampled_token_index] pos_tags.append(sampled_word)
        if sampled_word == '<eos>' or len(pos_tags) > max_decoder_seq_length:stop_condition =
            True
        target_sequence = np.zeros((1, 1)) target_sequence[0, 0] =
        sampled_token_indexstates_value = [h, c]
    return ' '.join(pos_tags)

# Test the model
for input_text in input_texts:
    input_seq = pad_sequences([[input_word2idx[word] for word in
input_text.split()]], maxlen=max_encoder_seq_length)
    predicted_pos_tags = generate_pos_tags(input_seq)print('Input:',
    input_text)
    print('Predicted POS Tags:', predicted_pos_tags)print()
```

OUTPUT

```
Epoch 1/50
1/1 [==============================] - 7s 7s/step - loss: 1.3736 - accuracy: 0.0000e+0
0 - val_loss: 1.1017 - val_accuracy: 0.0000e+00Epoch 2/50
1/1 [==============================] - 0s 63ms/step - loss: 1.3470 - accuracy: 0.7500
- val_loss: 1.1068 - val_accuracy: 0.0000e+00
Epoch 3/50
1/1 [==============================] - 0s 65ms/step - loss: 1.3199 - accuracy: 0.7500
- val_loss: 1.1123 - val_accuracy: 0.0000e+00
Epoch 4/50
Epoch 44/50
1/1 [==============================] - 0s 58ms/step - loss: 0.0882 - accuracy: 0.7500
:
```

:
:
Epoch 50/50
1/1 [==============================] - 0s 60ms/step - loss: 0.0751 - accuracy: 0.7500
- val_loss: 2.2554 - val_accuracy: 0.0000e+00

Input: I love coding
Predicted POS Tags: VB NNP NNP DT DT

Input: This is a pen
Predicted POS Tags: VBZ DT NN NN DT

Input: She sings well
Predicted POS Tags: VB NNP NNP DT DT

RESULT

Thus Parts of speech tagging using Sequence to Sequence architecture is implemented.

Experiment 7 **Machine Translation using Encoder-Decoder model**

Date

AIM:

To implement Machine Translation using Encoder-Decoder model

CONCEPT:

Machine translation using an Encoder-Decoder model is a popular approach in natural language processing (NLP) for automatically translating text from one language to another.

1. **Encoder**: The encoder is responsible for processing the input text in the source language and transforming it into a fixed-length representation called a "context vector" or "thought vector." This context vector is a condensed representation of the input text thatcaptures its essential information..
2. **Decoder:** The decoder takes the context vector generated by the encoder and generates the translated text in the target language. Like the encoder, the decoder is usually implemented using an RNN, LSTM, or transformer architecture.
3. **Training:** he model is trained on a parallel corpus of source and target language pairs. During training, it learns to minimize the translation error by adjusting the model's parameters. This is typically done using techniques like maximum likelihood estimation (MLE) or more advanced methods like teacher forcing.
4. **Inference:** During inference or translation, the model takes an input sentence in the source language, encodes it to obtain the context vector, and then decodes to generate the translated sentence in the target language.

STEPS

1	Define the input and output sequences.
2	Create a set of all unique words in the input and target sequences.
3	Add <sos> and <eos> tokens to target_words.
4	Define the maximum sequence lengths Create dictionaries to map words to integers.Define the maximum sequence lengths
5	Prepare the encoder input data Prepare the decoder input and target data
6	Define the encoder input and LSTM layersDefine the decoder input and LSTM layers
7	Define, Compile and train the model
8	Define the encoder model to get the encoder states Define the decoder model with encoder states as initial state

9 Define a function to perform inference and generate translationsTest the model

PROGRAM

```
import numpy as np import
tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, LSTM, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Define the input and output sequences
input_texts = ['I love coding', 'This is a pen', 'She sings well'] target_texts = ['Ich liebe das Coden', 'Das
ist ein Stift', 'Sie singt gut']

# Create a set of all unique words in the input and target sequencesinput_words = set()
target_words = set()
for input_text, target_text in zip(input_texts, target_texts):input_words.update(input_text.split())
    target_words.update(target_text.split())

# Add <sos> and <eos> tokens to target_words
target_words.add('<sos>') target_words.add('<eos>')

# Create dictionaries to map words to integers
input_word2idx = {word: idx for idx, word in enumerate(input_words)} input_idx2word = {idx:
word for idx, word in enumerate(input_words)} target_word2idx = {word: idx for idx, word in
enumerate(target_words)}target_idx2word = {idx: word for idx, word in enumerate(target_words)}

# Define the maximum sequence lengths
max_encoder_seq_length = max([len(text.split()) for text in input_texts]) max_decoder_seq_length =
max([len(text.split()) for text in target_texts])

# Prepare the encoder input data
encoder_input_data = np.zeros((len(input_texts), max_encoder_seq_length),dtype='float32')
for i, input_text in enumerate(input_texts):
    for t, word in enumerate(input_text.split()): encoder_input_data[i, t] =
        input_word2idx[word]

# Prepare the decoder input and target data
decoder_input_data = np.zeros((len(input_texts), max_decoder_seq_length),dtype='float32')
```

```
decoder_target_data = np.zeros((len(input_texts), max_decoder_seq_length,len(target_words)),
dtype='float32')
for i, target_text in enumerate(target_texts):for t, word in
    enumerate(target_text.split()):
        decoder_input_data[i, t] = target_word2idx[word]if t > 0:
            decoder_target_data[i, t - 1, target_word2idx[word]] = 1.0

# Define the encoder input and LSTM layersencoder_inputs =
Input(shape=(None,))
encoder_embedding = tf.keras.layers.Embedding(len(input_words),256)(encoder_inputs)
encoder_lstm = LSTM(256, return_state=True)
encoder_outputs, state_h, state_c = encoder_lstm(encoder_embedding)encoder_states = [state_h,
state_c]

# Define the decoder input and LSTM layersdecoder_inputs =
Input(shape=(None,))
decoder_embedding = tf.keras.layers.Embedding(len(target_words),256)(decoder_inputs)
decoder_lstm = LSTM(256, return_sequences=True, return_state=True)decoder_outputs, _, _ =
decoder_lstm(decoder_embedding, initial_state=encoder_states)
decoder_dense = Dense(len(target_words), activation='softmax')decoder_outputs =
decoder_dense(decoder_outputs)

# Define the model
model = Model([encoder_inputs, decoder_inputs], decoder_outputs)

# Compile and train the model
model.compile(optimizer='adam', loss='categorical_crossentropy',metrics=['accuracy'])
model.fit([encoder_input_data, decoder_input_data], decoder_target_data,batch_size=64,
epochs=50, validation_split=0.2)

# Define the encoder model to get the encoder states encoder_model =
Model(encoder_inputs, encoder_states)

# Define the decoder model with encoder states as initial statedecoder_state_input_h =
Input(shape=(256,)) decoder_state_input_c = Input(shape=(256,))
decoder_states_inputs = [decoder_state_input_h, decoder_state_input_c]decoder_outputs, state_h,
state_c = decoder_lstm(decoder_embedding, initial_state=decoder_states_inputs)
decoder_states = [state_h, state_c] decoder_outputs =
decoder_dense(decoder_outputs)
```

```
decoder_model = Model([decoder_inputs] + decoder_states_inputs, [decoder_outputs]
+ decoder_states)

# Define a function to perform inference and generate translationsdef
translate(input_sequence):
    states_value = encoder_model.predict(input_sequence)target_sequence
    = np.zeros((1, 1)) target_sequence[0, 0] = target_word2idx['<sos>']
    stop_condition = False
    translation = []
    while not stop_condition:
        output_tokens, h, c = decoder_model.predict([target_sequence] +states_value)
        sampled_token_index = np.argmax(output_tokens[0, -1, :])sampled_word =
        target_idx2word[sampled_token_index] translation.append(sampled_word)
        if sampled_word == '<eos>' or len(translation) > max_decoder_seq_length:stop_condition = True
        target_sequence = np.zeros((1, 1)) target_sequence[0, 0] =
        sampled_token_indexstates_value = [h, c]
    return ' '.join(translation)

# Test the model
for input_text in input_texts:
    input_seq = pad_sequences([[input_word2idx[word] for word in
input_text.split()]], maxlen=max_encoder_seq_length)
    translated_text = translate(input_seq) print('Input:',
    input_text) print('Translated Text:', translated_text)print()
```

OUTPUT

Input: This is a pen

Translated Text: ist ein Stift Coden ein

RESULT

Thus Machine Translation using Encoder-Decoder model is implemented.

Experiment 8 **Image augmentation using GANs**

Date

AIM:

To implement Image augmentation using GANs

CONCEPT:

Image augmentation using Generative Adversarial Networks (GANs) is a technique that leverages the power of GANs to generate new, realistic images that are variations of existing images. This approach is commonly used in computer vision tasks, such as image classification and object detection, to increase the diversity and size of training datasets.

1. **Generative Adversarial Networks (GANs)**:GANs consist of two neural networks: a generator and a discriminator.The generator network takes random noise as input and generates synthetic images.The discriminator network tries to distinguish between real and synthetic images. During training, the generator aims to produce images that are indistinguishable from real ones, while the discriminator tries to get better at telling themapart.
2. **Image Augmentation with GANs:** You train a GAN on this dataset, where the generator learns to generate images similar to those in the dataset, and the discriminator learns to distinguish real images from generated ones.
3. **Generating Augmented Images:** Once the GAN is trained, you can use the generator to create new, synthetic images. To augment an image from your dataset, you feed it tothe generator, and the generator produces a new image. These generated images aretypically variations of the original images, introducing changes in aspects like style, lighting, perspective, or other factors that the GAN has learned from the training data.

STEPS

1 Load the MNIST dataset
Normalize and reshape the imagesDefine the generator network

2 Define the discriminator network
Compile the discriminator
Combine the generator and discriminator into a single GAN model

3 Train the hyperparameters and the Training loop has the following steps:

- Generate a batch of fake images
- Train the discriminator
- Train the generator
- Print the progress and save samples

PROGRAM

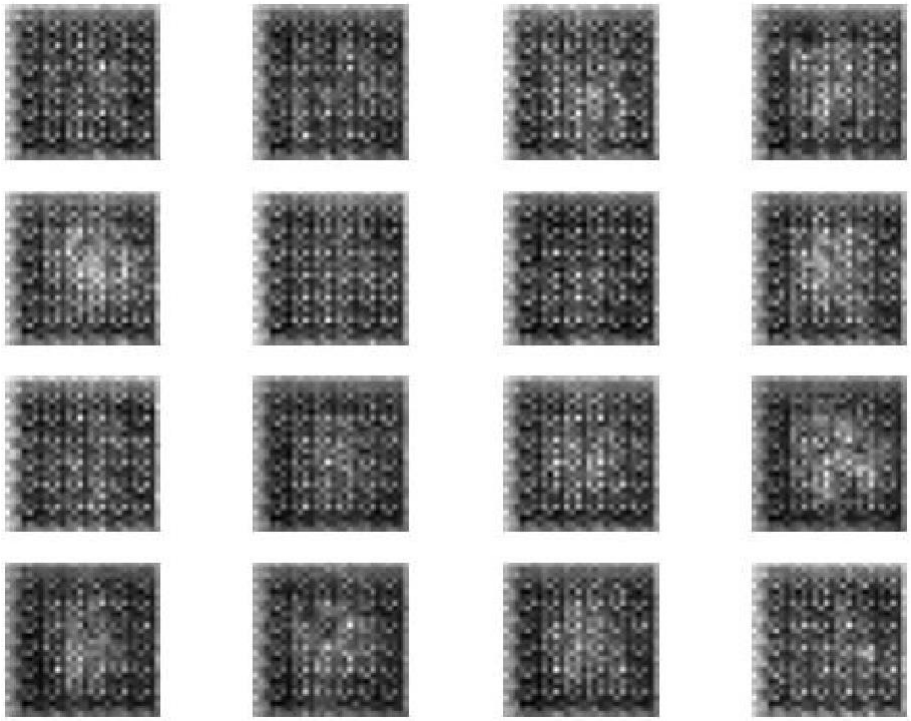
```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.layers import Input, Dense, Reshape, Flattenfrom
tensorflow.keras.layers import BatchNormalization, Dropout from tensorflow.keras.layers
import Conv2D, Conv2DTranspose
from tensorflow.keras.models import Sequential, Modelfrom
tensorflow.keras.optimizers import Adam

# Load the MNIST dataset
(x_train, _), (_, _) = mnist.load_data()

# Normalize and reshape the images
x_train = (x_train.astype('float32') - 127.5) / 127.5x_train =
np.expand_dims(x_train, axis=-1)

# Define the generator networkgenerator
= Sequential()
generator.add(Dense(7 * 7 * 256, input_dim=100))
generator.add(Reshape((7, 7, 256)))generator.add(BatchNormalization())
generator.add(Conv2DTranspose(128, kernel_size=5, strides=1, padding='same',activation='relu'))
generator.add(BatchNormalization())
generator.add(Conv2DTranspose(64, kernel_size=5, strides=2, padding='same',activation='relu'))
generator.add(BatchNormalization())
generator.add(Conv2DTranspose(1, kernel_size=5, strides=2, padding='same',activation='tanh'))

# Define the discriminator network
discriminator = Sequential()
discriminator.add(Conv2D(64, kernel_size=5, strides=2, padding='same',input_shape=(28, 28, 1),
activation='relu')) discriminator.add(Dropout(0.3))
discriminator.add(Conv2D(128, kernel_size=5, strides=2, padding='same',activation='relu'))
discriminator.add(Dropout(0.3))
```

```
discriminator.add(Flatten()) discriminator.add(Dense(1,
activation='sigmoid'))

# Compile the discriminator discriminator.compile(loss='binary_crossentropy',
optimizer=Adam(learning_rate=0.0002,  beta_1=0.5),  metrics=['accuracy'])

# Combine the generator and discriminator into a single GAN modelgan_input =
Input(shape=(100,))
gan_output  =  discriminator(generator(gan_input))gan =
Model(gan_input, gan_output)
gan.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=0.0002,beta_1=0.5))

# Training hyperparametersepochs
= 100
batch_size = 128
sample_interval = 10

# Training loop
for epoch in range(epochs):
      # Randomly select a batch of real images
      idx = np.random.randint(0, x_train.shape[0], batch_size)real_images =
      x_train[idx]

      # Generate a batch of fake images
      noise = np.random.normal(0, 1, (batch_size, 100))fake_images =
      generator.predict(noise)

      # Train the discriminator
      x = np.concatenate((real_images, fake_images))
      y = np.concatenate((np.ones((batch_size, 1)), np.zeros((batch_size, 1))))d_loss =
      discriminator.train_on_batch(x, y)

      # Train the generator
      noise = np.random.normal(0, 1, (batch_size, 100))
      g_loss = gan.train_on_batch(noise, np.ones((batch_size, 1)))

      # Print the progress and save samplesif epoch %
      sample_interval == 0:
            print(f'Epoch: {epoch}              Discriminator Loss: {d_loss[0]}                  Generator
Loss: {g_loss}')
            samples = generator.predict(np.random.normal(0, 1, (16, 100)))samples = (samples *
            127.5) + 127.5
            samples = samples.reshape(16, 28, 28)fig, axs =
            plt.subplots(4, 4)
```

```
count = 0
for i in range(4):
    for j in range(4):
        axs[i, j].imshow(samples[count, :, :], cmap='gray')axs[i, j].axis('off')
        count += 1plt.show()
```

OUTPUT

Epoch: 90 Discriminator Loss: 0.03508808836340904Generator Loss: 1.736445483402349e-06



RESULT

Thus **Image augmentation using GANs** is implemented.

Experiment 9 MINI-PROJECT ON REAL WORLD APPLICATIONS (task manager)

Date:

AIM:

To mini project on real world application to create task manager.

PROGRAM:

```
Import os
    Import datetime

    Def load_tasks(file_path):
        Tasks = []
        If os.path.exists(file_path):
            With open(file_path, 'r') as file:
                Lines = file.readlines()
                For line in lines:
                    Task, status, date_str = line.strip().split(',')
                    Date = datetime.datetime.strptime(date_str, '%Y-%m-%d').date() if date_str else None
    Tasks.append({'task': task, 'status': status, 'date': date})
        Return tasks

    Def save_tasks(file_path, tasks):
        With open(file_path, 'w') as file:
            For task in tasks:
    Task_str = f'{task['task']},{task['status']},{task['date']}\n"
    File.write(task_str)

    Def add_task(tasks, new_task):
    Tasks.append({'task': new_task, 'status': 'Incomplete', 'date': datetime.date.today()})
        Print(f"Task '{new_task}' added successfully.")
```

```
Def complete_task(tasks, task_index):
    If 1 <= task_index<= len(tasks):
        Tasks[task_index – 1][‘status’] = ‘Completed’
        Tasks[task_index – 1][‘date’] = datetime.date.today()
        Print(f”Task ‘{tasks[task_index – 1][‘task’]}’ marked as completed.”)
    Else:
        Print(“Invalid task index.”)

Def view_tasks(tasks):
    If not tasks:
        Print(“No tasks found.”)
    Else:
        Print(“Tasks:”)
        For I, task in enumerate(tasks, start=1):
            Print(f”{i}. {task[‘task’]} – {task[‘status’]} ({task[‘date’]})”)

Def view_summary(tasks):
Completed_tasks = [task for task in tasks if task[‘status’] == ‘Completed’]
Incomplete_tasks = [task for task in tasks if task[‘status’] == ‘Incomplete’]

    Print(f”\nSummary:”)
    Print(f”Total tasks: {len(tasks)}”)
    Print(f”Completed tasks: {len(completed_tasks)}”)
    Print(f”Incomplete tasks: {len(incomplete_tasks)}”)

Def main():
File_path = ‘tasks.txt’
    Tasks = load_tasks(file_path)
```

```
    While True:
        Print("\n1. Add Task\n2. Complete Task\n3. View Tasks\n4. View Summary\n5. Quit")
        Choice = input("Enter your choice (1/2/3/4/5): ")


        If choice == '1':
New_task = input("Enter the task: ")
Add_task(tasks, new_task)
Elif choice == '2':
View_tasks(tasks)
Task_index = int(input("Enter the task number to mark as completed: "))
Complete_task(tasks, task_index)
Elif choice == '3':
View_tasks(tasks)
Elif choice == '4':
View_summary(tasks)
Elif choice == '5':
Save_tasks(file_path, tasks)
            Print("Exiting. Tasks saved.")
            Break
        Else:
            Print("Invalid choice. Please enter a valid option.")


If __name__ == "__main__":
    Main()
```

OUTPUT:

1. Add Task
2. Complete Task
3. View Tasks
4. View Summary
5. Quit

Enter your choice (1/2/3/4/5): 1

Enter the task: Complete Python mini-project

Task ‘Complete Python mini-project’ added successfully.

1. Add Task
2. Complete Task
3. View Tasks
4. View Summary
5. Quit

Enter your choice (1/2/3/4/5): 2

Tasks:

1. Complete Python mini-project – Incomplete (2023-02-21)

Enter the task number to mark as completed: 1

Task ‘Complete Python mini-project’ marked as completed.

1. Add Task
2. Complete Task
3. View Tasks
4. View Summary
5. Quit

Enter your choice (1/2/3/4/5): 3

Tasks:

1. Complete Python mini-project – Completed (2023-02-21)

1. Add Task
2. Complete Task
3. View Tasks
4. View Summary
5. Quit

Enter your choice (1/2/3/4/5): 4

Summary:

Total tasks: 1

Completed tasks: 1

Incomplete tasks: 0

1. Add Task
2. Complete Task
3. View Tasks
4. View Summary
5. Quit

Enter your choice (1/2/3/4/5): 5

Exiting. Tasks saved.

RESULT: Thus, the Mini-Project On Real World Applications (task manager) implemented successfully.